

CS – 432 Databases

Assignment – IV

The GitHub Link to access the project content is given below: https://github.com/AryanMeshram059/db_4

The link to the YouTube video containing explanation of Sharding of our database is:

https://drive.google.com/file/d/1V9_C5dvVIUGZypgvttP7Ujz5aopBg0gy/view?usp=sharing

Part – I: Scalability of Database

Introduction to Sharding: Horizontal Vs Vertical Scaling:

So, as we saw in our all three previous assignments, we mainly focused on:

1. **Database Creation:** SQL Database of Attendance Management Portal
2. **B+ Tree Indexing:** We learnt what is B+ Tree and why is it used. Then we also saw its various operations, understood its implementation and compared it even with Brute Force DB to show why B+ tree is the King.
3. **Application Development:** Then we converted our whole database into an application platform.
4. **ACID Properties:** Then, we moved to implemented transaction manager, WAL etc. in our API as well as our B+ Tree so that it can satisfy all ACID properties.

But, no where we focused on one of the most important aspect that every user wants: Speed!

A database can easily have millions, billions of entries and as database size increases, the query time may also increase which may slow down its speed. Larger datasets increase system load and storage requirements, which may affect performance without proper scaling. And we seriously don't want it! So what our database that has a fully-developed schema, ACID verifiable lacks are nothing else but **Scalability**.

So in this assignment, we are going to tackle this issue and see how we can make our database to query fast. So let's start with Scalability.

→ Scalability: Two Paths to Speed:

Whenever a database becomes massive, it starts to take more time to run queries like search, delete, range query etc. Hence, the database becomes slow because of holding millions of records. Even in our case, if we see our attendance table, with just 15 students, there were above 350+ entries. And now imagine a school/college with thousands of children, clearly the database would become overload.

So to fix this issue, we use Scalability. **Scaling** refers to increasing the capacity of a system to handle **larger workloads, more data, or more users**. In database systems, scaling can be done in two main ways.

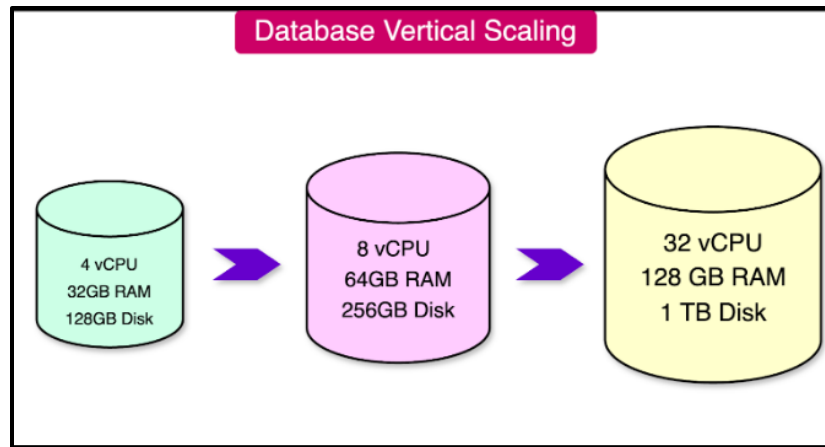
1. **Vertical Scaling**
2. **Horizontal Scaling**

☐ Vertical Scaling:

Vertical Scaling means adding more resources like CPU, RAM, Storage to a single server to increase its capacity. Hence, as the data size increases, to accommodate that we upgrade our hardware by adding better resources to our server.

Hence, this is called **Scale Up**. So by adding more CPU, RAM, storage we can easily store and query database faster.

Advantage: So its advantage is very trivial! Its very easy to implement. We don't need to modify our SQL or API logic, only modify at hardware (physical) level.



Reference: [A Crash Course in Database Scaling Strategies](#)

Disadvantage: One of the major disadvantage of this method is that there is always a **Hardware Ceiling**. There is only limited amount of RAM that we can put on a single motherboard.

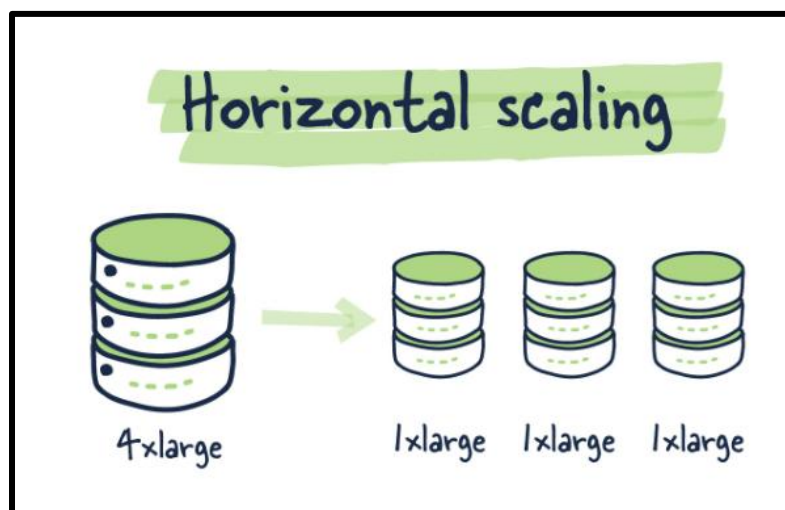
And not only this, one of the critical disadvantage is **Single Point of Failure**. If one powerful server dies, then whole database collapses.

Hence, due to these vertical scaling is not always the good option. And hence, that's why we have: **Horizontal Scaling**.

□ Horizontal Scaling:

Upgrading only a single server clearly didn't work. Now, the main problem is because we have so many data and only one server is handling a lot. So why not **divide and conquer**? That's what horizontal scaling is about.

Horizontal scaling is the process of increasing the capacity of database by adding more and more servers instead of upgrading a single one. Hence, this distributes the load across a cluster of servers. And this is where our assignment: Sharding comes in!



Reference: [Database Sharding Explained - by Mahdi Yusuf](#)

One of the main advantage of this method is horizontal scaling supports much larger scalability than vertical scaling. Even if the data grows from thousand to lakh, still we can just add more shards by modifying the strategy through which we created it.

The main drawback is that query routing will now become more complex and also data partitioning is complex — joins and transactions across shards are hard to implement efficiently. Consistency models in RDBMS are not naturally distributed; ensuring strong consistency across nodes adds latency and complexity.

But if we have a huge scale data and high availability, then Horizontal Scaling definitely works a lot better than vertical scaling.

So, in short:

- We use vertical scaling for simplicity and when growth is moderate
- We adopt horizontal scaling when we need massive scalability and high availability.

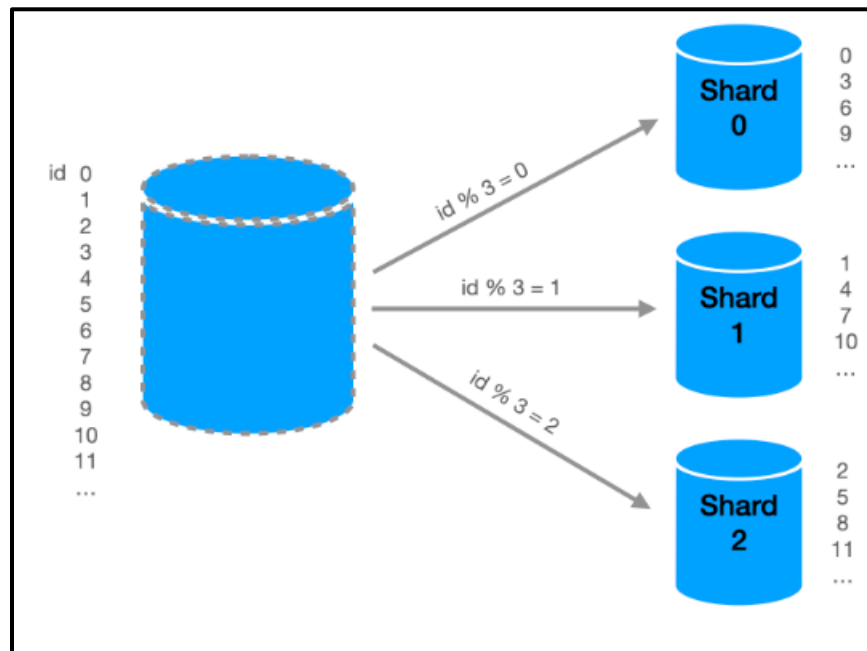
And in this assignment, we are going to see one of the most famous way to do horizontal scaling → **Sharding**.

→ Sharding:

Sharding is a database architecture technique used to achieve horizontal scaling by distributing the large data set across multiple database serves which are smaller, faster, and much more easily manageable. These multiple chunks are what we call **Shards**.

Unlike standard partitioning—which usually happens within a single database engine—sharding distributes these chunks across entirely separate database instances. This creates a **Shared-Nothing Architecture**, where each shard is a self-contained universe with its own CPU, RAM, and storage.

Hence, instead of storing entire dataset into a single database server, data is partitioned into smaller subsets and each subset is stored in its allocated shard. Each shard contains only a portion of the total database, but together all shards represent the complete database.



Reference: [Database Partitioning in System Design](#)

This allows the workload of storing and querying data to be distributed across multiple servers, improving performance and scalability.

□ Shard Key:

In a sharded database system, a specific attribute from the database is chosen which we call the **shard key** and now this is used to determine which shard a particular record should be stored in.

The shard key must be chosen carefully so that the data is distributed evenly across shards, preventing any single shard from becoming overloaded. There are various parameters on which a shard key is chosen:

1. **High Cardinality**
2. **Query Aligned**

3. Stable

And then basis on this shard key we perform various partitioning strategies like:

1. **Hash Based Partition:** So one of the way to split database is to create a hash function and then rapidly use it. For example:

Shard key%10 can be such hash function according to which we create shards.

Hence, wherever value is 0, it goes to shard0, if 1 then shard1 ...

2. **Range Based Partition:** Another way can be to find the total number of data in dataset and then allocate equally spaced data into different shards.

For eg: If we have 100 dataset, then I can shard like:

0,1,...10, → shard0

11,12,..., 20 → shard 1 ...

3. **Directory Based Partition:** The third way to partition database is to use a lookup table (like a map in C++). So in this we create a map or table that maps data to a shard:

For eg:

- Student 230001 → Shard 0
- Student 230005 → Shard 2
- Student 240001 → Shard 1

Every time the API wants to find a student, it doesn't do math; it asks the "Phone Book" table first, gets the location, and then goes there. Hence depending on range we partition data.

Hence, sharding simply means to distribute the single database server across different servers called shards so that we can scale horizontally. So, now we have good idea of what is Sharding, Horizontal Scaling and Vertical Scaling. Now let's now go to our main report:

Sharding Vs Upgrading Single Database Server:

So, upgrading a single database server means Vertical Scaling and sharding means horizontal scaling. The sharding mechanism is a lot different from upgrading single database server and lot better.

Let's understand this through the following difference table:

Feature	Vertical Scaling (Upgrading Single DB Server)	Horizontal Scaling (Sharding)
Method	We add more hardware resources like RAM, CPU, SSD (storage) to the existing server to make it accommodate huge data and run fast.	We add more servers called shards and then distribute the data of main database across various servers as per our partitioning strategy on shard key.
Architecture	It is a single database server: A single machine	It has multiple databases called shards.
Complexity	Very low. Simple to implement and no changes needed to SQL and API.	Very High as it requires more query routing logic in API and data migration
Scalability	Hard Hardware Ceiling: We may eventually run out of upgrade options. Hence it is limited by hardware.	Highly scalable as we can easily add more shards as data increases. It is not theoretically limited.
Reliability	Single Point Failure: If server dies, database is gone.	Fault Isolation: If one shard dies, the others are still surviving. Hence data isn't completely lost.

Thus, while vertical scaling improves the capacity of a single machine, sharding enables distributed scalability by spreading the data and workload across multiple independent database nodes.

Part – II: The Shards of Our Database

🔗 Choice of Shard Key:

So for our Attendance Management Portal, the shard key that we chose was:

Student_id

Now, one of the main questions that we had during this was why student_id? Not member_id? As if we see then we had just 15 students, but 35 members. But after careful analysis, we got major points that why student id and not member id:

→ Query Pattern:

One of the aspects to select a shard key is: **Query Aligned**. That means we should pick the attribute, which is most often used to get data, i.e., most often used in where clause.

In an attendance management portal, most of the queries are like:

- Get attendance % of student in multiple courses at id = S
- Get the courses the student of id = S enrolled in
- Get the attendance requests of the students at given ids
- Find the students who are below the threshold.

So clearly, most of our queries in attendance are related to students. Hence, Student ID appears in most of the queries.

So, if we made student id as shard key, then API can:

- Directly route to one shard using student id and applying strategy we used.
- And hence no need to search across shards.

Since, one of the most used attribute in almost all queries is Student id, we chose that as our shard key.

→ Data Locality:

The largest tables in our database are:

1. Attendance (385 rows)
2. Lecture log (127 rows)
3. Enrollment (63 rows)

And out of these three, two of the tables: attendance and enrollment query using the student id as of course these are student based records.

Along with this, we have other tables like attendance_request, notification, which also uses student id a lot.

Hence, Sharding by student ID groups the related data together. Hence, for a student id we just need to go to the assigned shard and all his details will be available there. This reduces cross-shard joins!

→ Cardinality and Distribution:

Another major aspect of choosing student id is because a shard key must have high cardinality. In our database, though we have only 15 students, but if we see the relative cardinality wrt other tables, in attendance we have 385 rows and each have a corresponding student id. Similarly for enrollment, attendance request etc. we have a student id corresponding to them.

In short, overall, if we see the whole database, student_id is the only id which has been used maximum number of times in tables and hence, has highest cardinality.

Hence, student id having cardinality is optimal choice for the Shard key as each student has a unique identifier and the as the system scales, number of students can grow significantly.

→ Stable:

Once a student id is registered, then until the year or semester ends, its entry cannot be removed from database. Hence, student id is very stable as adding any record doesn't affect student id. It doesn't change frequently when a record is inserted.

□ Why not member_id?

One of the main problems with member id is:

How to distribute?

If we use some hash function, then student, teacher, and admin data all gets mixed up. There's no systematic table and hence no meaningful partition.

If we distribute data as per admin, teacher and student; then our largest table: Attendance, enrollment remains unchanged for in the server where student is stored, the whole large table is needed.

In this case, one shard would be very heavy loaded while other two would be too light loaded, which affects the balance of our database. There was no advantage in splitting with member_id. Same issues for teacher_id, course_id etc.

Hence, considering all facts that student_id is query-optimum, high cardinality, stable, future-scalable and important in largest of tables: we chose our shard key as **student id**.

🌀 Partitioning Strategy:

Once we have chosen our shard key: the next step comes to choose **Partitioning Strategy**. So the portioning strategy that we used in our database is:

Hash – Based Partitioning Strategy

So, before seeing why hash based, lets see why not other two?

→ Why not Range Based?

One of the main reasons for not doing range based is imbalanced database.

Suppose we used range based like:

All students of 23xxxx → Shard0

All students of 24xxxx → Shard1

But then, it is easily possible that there are huge number of registrations in 2024 batch as compared to 2023. Say, in 2023 we had 150 but in 2024, it went up to 350-400. Clearly, in such case; one shard would be very heavily loaded which might reduce its speed but other is still normal.

Then to correct it, we would need to again shard 2024 batch and this creates more shard. And we know from Part – I that as shards increases, the API logic as well as database query routing becomes very complex.

Hence, clearly Range partitioning may lead to load imbalance across shards if the data distribution is skewed, especially when certain ID ranges contain significantly more records. So what we want is that data to be distributed equally within all shards, not that some are fast while some slow. There must be uniformity.

Also another reason not to use this strategy is: Range based strategy is used more often for queries where we need to extract data of a range of students say:

Students between ids 23xxxx → 23yyyy

But in attendance management portal, most of the queries are on students itself, not ranges. Hence, this strategy gives us no advantage.

→ Why not Directory Based?

So, we didn't also use directory based partitioning as:

1. **Double-Hop Problem:** With Directory-based, every query becomes two queries as first, we have to query the directory to find the shard, and then we query the shard itself. This adds latency (slowness).
2. **Bottleneck:** The directory service can become a **bottleneck** or **single point of failure** if not properly replicated or distributed. Hence, if the Directory table is slow or crashes, the entire system—all shards—become unreachable because no one knows where to go.
3. **Management Overhead:** As every time a new student is added, we have to manually update the Directory every time. Hence, its lot of human work.

Hence, rejecting these two, we came to the strategy that was easiest to implement, had least load, and had ability to fill all the cups equally!

→ Hash Based Partitioning:

So, we know all student numbers irrespective of years are always of form:

230000, 230001, 230002, ...

240000, 240001, 240002, ...

Now, to equally distribute this among shards, we can use the hash based partitioning with the hash function as:

(Student_id % n)

Where n is the number of shards we create.

Since, in our API and SQL database, we created three shards our hash function was:

Student_id % 3

This is the best of all three partitioning strategy for our database because:

1. **Even Distribution:** Now, the possible results of $\text{id} \% 3$ are:

0, 1 and 2

Hence, depending on these results, the shards can be created.

Since student ids or roll numbers are always consecutive numbers, it is guaranteed that:

If current id goes to shard0, then next id must go to 1 and after that to 2 and cycle repeats.

Hence, because student IDs are consecutive numbers, the modulo operation distributes records approximately evenly across shards, leading to balanced load. So we won't have one shard heavily loaded and other lightly loaded. It would be balanced.

2. **Easy to Implement:** With hash function like:

Student id % 3

We can easily implement it on API as now we just need to search shard using the modulo value and then we can simply find the shard. No double query nor search!

And if the system scales further, additional shards can be added and the hash function updated accordingly, followed by redistribution of data across shards which can be done easily.

And if we see real databases like:

- User databases
- Social networks
- Attendance systems

All use hash based sharding on user id so that user data can be evenly spread. That's why we picked hash based strategy as its easy to implement, distributes data evenly and one of the most common strategy in real databases.

🌀 API and Query Routing:

🌀 Which SQL shard tables were created and how data was migrated:

To shard our database, on basis of student ID, we chose a subset of the database tables to perform horizontal scaling. The tables that we chose for sharding were majorly the ones that contained large and frequently growing datasets and especially they were related to **student ID**.

So the tables that we chose to shard were:

1. Student
2. Enrollment
3. Attendance
4. Attendance_request
5. Notification

This is because all these tables contained student ID and could easily grow very large as number of students increased. Hence, for horizontal scaling we chose these tables. While rest tables:

1. Member
2. Teacher
3. Department
4. Teacher Department
5. Admin
6. Audit Log
7. Course
8. Course Slot
9. Lecture Log

Were replicated exactly. It's clear for 2 – 9 tables why they were replicated as they didn't have anything to do with student. But question comes why not member?

→ Why not member?

Member table, which also had student ID, was not sharded. The main reasons is because it stores **global identity information** used across the system, i.e., it includes data of all teachers, students as well as admins. Many parts of the system rely on this table for authentication and user identification.

- **Authentication Bottleneck:** When a user enters their credentials in API, the system doesn't yet know if they are a Student, Teacher, or Admin.
If it were sharded, then the API would have to broadcast a query to every single shard just to find where that user lives.
By keeping it global, any shard can handle the login immediately. This is called **Decoupled Authentication**.
- **Imbalanced Sharding:** Along with this, if this was sharded, then it would be like partial sharding. As one part of table (teacher, admin) is in each shard but students are splitted. It won't be a balanced sharding.
Since Teachers and Admins don't follow the Student ID pattern, sharding them would lead to **Data Fragility**. If we split them, we would have "orphaned" records that don't fit the modulo logic. Keeping them global ensures **Referential Integrity**. Hence, we kept member as a global table (central) and all other tables depend on it.
- **Metadata:** This might also be helpful if in case a shard got failed, instead of losing data of certain students, we can retrieve it back as member table of all shards would have data of all students. This means in a database

system, even if a shard goes down, though we lose the attendance for those students, but because the member table is replicated, the system still knows those students exist due to presence of other shards. It allows for Graceful Degradation—the system can still show user profiles even if the specific transactional data is temporarily offline.

Hence, that's why we didn't shard member and replicated them across all shards.

→ Data Migration:

So to migrate the data, we did the following steps:

1. We created three additional databases (shards) named: attendance_shard0, attendance_shard1, attendance_shard2.
2. Then, we needed to copy the whole schema of original database: attendance_management_db to these shards. So firstly we created the dump file of only schema of original db by:

```
mysqldump -u root -p --no-data --set-gtid-purged=OFF attendance_management_db > C:\Users\HP\schema.sql
```

Then this created a schema.sql file that contains only schema of our original database. (only database structure (no data)).

After that, we need to add this schema to the three databases we created. Hence, then we executed the following instruction and got schema into three databases:

```
mysql -u root -p attendance_shard0 < C:\Users\HP\schema.sql
mysql -u root -p attendance_shard1 < C:\Users\HP\schema.sql
mysql -u root -p attendance_shard2 < C:\Users\HP\schema.sql
```

3. Once the schema was copied, then it was just insertion from main database into three shards using the following hash function:
 - Student_id %3 == 0 → shard0
 - Student_id %3 == 1 → shard1
 - Student_id %3 == 2 → shard2

Then the tables we mentioned above, we used these hash functions to add data into table using following MySQL Query:

```
INSERT INTO student SELECT * FROM attendance_management_db.student WHERE (StudentID % 3) = 0;
```

(Here we had used attendance_shard0)

This was done for each shard. This ensured that each record was stored in exactly one shard based on its student identifier.

And for table that were exactly replicated, we used following query:

```
INSERT INTO member SELECT * FROM attendance_management_db.member;
```

And repeated this for all three shards.

4. By this way, we migrated all our data from original database into all three shards.

So, this was all about Sharding tables and data migration.

→ Network Configuration: Local Hosting the Shards:

Once the shards were made in the SQL database, then we hosted them to the local server using the phpMyAdmin. So to simulate a distributed database system, all three shards were locally deployed on separate MySQL instances running on different ports: 8081, 8082, 8083.

To access them, we can use the following links:

1. **Shard 0 (Port 8081):** <http://10.0.116.184:8081/>
2. **Shard 1 (Port 8082):** <http://10.0.116.184:8082/>
3. **Shard 2 (Port 8083):** <http://10.0.116.184:8083/>

This system was then tested in a local environment by executing application-level queries and verifying data distribution across shards. Manual validation was performed using MySQL queries on MySQL CLI to ensure that records were correctly routed based on the hashing function (`StudentID % 3`).

This setup allowed simulation of a distributed sharded architecture without requiring external servers. Hence, by this way we implemented sharding in our database.

🌀 Sharding Approach: Multiple Database Instances:

So, we covered the major aspect of how we sharded our database in previous section. But still let's quickly see what we did in sharding:

Our approach to sharding was based on **multiple independent database instances (shards)** and distributing data across these shards using **hash based partitioning strategy** on the shard key: **student_id**.

This process included:

1. **Schema Duplication:** So firstly, we exported the schema from the original `attendance_management_db` using `mysqldump` and importing it into three shard databases (`attendance_shard0`, `attendance_shard1`, `attendance_shard2`).
2. **Data Partitioning:** Then we migrated student-centric tables (`student`, `enrollment`, `attendance`, `attendance_request`, `notification`) into shards using the rule `student_id % 3`. This ensured even distribution of records across shards.
3. **Replication of Global Tables:** Then we just copied the remaining reference and metadata tables (`member`, `teacher`, `department`, `course`, etc.) into all shards to maintain consistency and avoid authentication bottlenecks.
4. **Deployment:** Then we hosted each shard on separate local MySQL instances running on different ports (8081, 8082, 8083) to simulate a distributed environment. (as shown in previous section)
5. **Query Routing:** Application logic computes `student_id % 3` to determine the correct shard, ensuring queries are directed efficiently without scanning all shards.

This approach allowed us to achieve horizontal scalability while maintaining balanced load distribution, fault isolation, and efficient query performance.

➔ Validation of Sharding Correctness

So let's verify our sharding using the MySQL Command Line Client. So we will focus on our two main candidate tables:

❑ Student:

So in our original database, we had 15 rows.

We can show it using the following query:

```
Select * from student;
```

Using original database.

```
mysql> use attendance_management_db;
Database changed
mysql> select * from student;
```

StudentID	Roll_no	DepartmentID	BatchYear	CPI
230005	23CS001	1	2023	8.40
230008	23PH001	9	2023	7.90
230011	23CS002	1	2023	9.10
230014	23AI001	2	2023	8.20
230015	23EE001	3	2023	7.60
240001	24CS001	1	2024	8.80
240004	24MAT001	8	2024	8.00
240006	24EE001	3	2024	7.50
240012	24MA001	10	2024	8.60
240015	24CE001	7	2024	7.80
250003	25CS001	1	2025	8.90
250007	25CH001	6	2025	8.10
250009	25ICDT001	4	2025	7.40
250010	25PH001	9	2025	8.30
250013	25AI001	2	2025	7.90

```
15 rows in set (0.371 sec)
```

Now, if we see the data of students in sharded tables:

1. Shard0:

```
mysql> use attendance_shard0;
Database changed
mysql> select * from student;
```

StudentID	Roll_no	DepartmentID	BatchYear	CPI
240006	24EE001	3	2024	7.50
240012	24MA001	10	2024	8.60
240015	24CE001	7	2024	7.80

```
3 rows in set (0.006 sec)
```

Clearly, we have only three entries:

- 240006 (%3 = 0)
- 240012 (%3 = 0)
- 240015 (%3 = 0)

Hence, clearly our sharding works perfectly!

2. Shard1:

```
mysql> use attendance_shard1;
Database changed
mysql> select * from student;
```

StudentID	Roll_no	DepartmentID	BatchYear	CPI
230005	23CS001	1	2023	8.40
230008	23PH001	9	2023	7.90
230011	23CS002	1	2023	9.10
230014	23AI001	2	2023	8.20
240001	24CS001	1	2024	8.80
240004	24MAT001	8	2024	8.00
250003	25CS001	1	2025	8.90
250009	25ICDT001	4	2025	7.40

```
8 rows in set (0.011 sec)
```

So here, we have 8 entries and if we check all have:

StudentID %3 = 1

3. Shard2:

```
mysql> use attendance_shard2;
Database changed
mysql> select * from student;
+-----+-----+-----+-----+-----+
| StudentID | Roll_no | DepartmentID | BatchYear | CPI |
+-----+-----+-----+-----+-----+
| 230015 | 23EE001 | 3 | 2023 | 7.60 |
| 250007 | 25CH001 | 6 | 2025 | 8.10 |
| 250010 | 25PH001 | 9 | 2025 | 8.30 |
| 250013 | 25AI001 | 2 | 2025 | 7.90 |
+-----+-----+-----+-----+-----+
4 rows in set (0.012 sec)
```

And hence, here we have:

- $230015 \% 3 = 2$
- $250007 \% 3 = 2$
- $250010 \% 3 = 2$
- $250013 \% 3 = 2$

Hence, clearly our database does sharding perfectly.

And now if check total row count:

$$\text{Shard0 (3) + shard1 (8) + Shard2 (4) = 15}$$

Which clearly matches with the original database. Hence, we didn't lose any data and all students went to correct corresponding shards.

□ Attendance:

The largest table in our database and that we sharded! Since its difficult to show whole table for all 4, but if we just verified honestly the:

$$\text{Row count of Original} = \text{Sum of row count of all shards}$$

Then this would be enough to prove that our database is perfect.

So, row count in original database is:

```
mysql> use attendance_management_db;
Database changed
mysql> select count(*) from attendance;
+-----+
| count(*) |
+-----+
| 385 |
+-----+
1 row in set (0.076 sec)
```

Hence, it is 385.

For shards:

1. Shard0:

```
mysql> use attendance_shard0;
Database changed
mysql> select count(*) from attendance;
+-----+
| count(*) |
+-----+
|      79 |
+-----+
1 row in set (0.063 sec)
```

So clearly, here row count is 79.

2. Shard1:

```
mysql> use attendance_shard1;
Database changed
mysql> select count(*) from attendance;
+-----+
| count(*) |
+-----+
|     201 |
+-----+
1 row in set (0.061 sec)
```

So clearly, here row count is 201.

3. Shard2:

```
mysql> use attendance_shard2;
Database changed
mysql> select count(*) from attendance;
+-----+
| count(*) |
+-----+
|     105 |
+-----+
1 row in set (0.061 sec)
```

And here row count is 105.

So now if we find total sum:

$$\text{Shard0 (79) + Shard1 (201) + Shard2 (105) = 385.}$$

And clearly this matches the original count of attendance from original database.

Hence, this shows we didn't lose even a single record!

→ How Shard Isolation was Achieved:

So, shard isolation means that one shard doesn't affect the another shard, i.e., they're independent. So this was ensured by the way we made our shards:

1. **Separate Database Instances:** We built three independent databases: attendance_shard0, attendance_shard1 and attendance_shard2. Each has its own schema, tables and storage – no overlap nor shared table/files.
2. **Independent Hosting:** Each shard was hosted on a separate MySQL instance running on different ports (8081, 8082, and 8083). This means queries to one shard cannot accidentally “spill over” into another — they are isolated at the network level.

3. **Hash based Partitioning:** This is the root reason why its impossible for shards to affect one another. The shard key (Student ID % 3) ensures that each student's data is routed to exactly one shard. No one student record can be ever duplicated across shards, as student id % 3 can't have two values: its one-one function. So there's no overlap or cross-contamination across shards.
4. **Query Routing Logic:** The application/API computes the shard location before executing queries. This prevents queries from scanning multiple shards unnecessarily, keeping each shard isolated in terms of workload.

Hence, by this way our database sharding ensures isolation.

Part – III: The CAP of Our Database

☞ CAP Theorem:

According to CAP Theorem, (also known as Brewer's Theorem), whenever we create a distributed data system, i.e., we form shards or partition it, then it can guarantee **only two out of three properties** at a time:

1. **Consistency(C):** Every read receives the most recent write. This signifies all nodes see the same data at same time.
For example: In our sharded database, if we update attendance in shard1 and someone immediately runs a query to see the updated shard, they see the updated instantly.
A better example is **Profile Photo**. I kept the profile photo of say "Lies of P" on my WhatsApp. Then as soon as I change it, I must see it immediately. It shouldn't take much time to update otherwise this means database is working slowly and hence not consistent.

Hence, it should either update immediately or fail instead of returning old value.

Therefore, as soon as we update, we should be able to see it or it should throw error. That's consistency. If it showed old data, it means its not consistent.

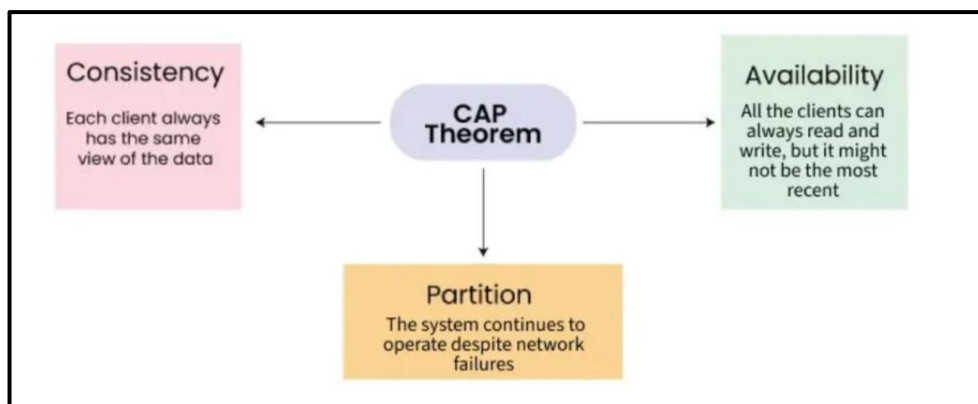
2. **Availability(A):** Every request receives a response even if some nodes are down or unreachable. This means that system is always active (available) even when there are technical issues inside it (in shards).
For example: Suppose our shard1 is not responding, then if a student from shard1 tries to see his attendance, then it must not be like the API shows wrong user or user not available.
It must atleast return the old data even if it's wrong. Hence, it must not ghost him by ignoring his query and returning nothing. If it can't response to user query, it means it's not available.

If it can just recognize but can't answer, that's **partial availability**. So availability just wants response though incorrect.

3. **Partition Tolerance(P):** The system continues to operate even if network between two shards breaks.

For example:

If connection between shard0 and shard1 gets failed, then still database can operate independently of it with help of other shards.



We have another term called **Fault Tolerance**:

4. **Fault Tolerance:** The system continues to operate even when some of its components fail. The system is designed so that failures do not stop the entire service. A fault can be:
 - server crash (shard failure)
 - Database failure
 - Disk failure

- Software bug

A fault-tolerant system detects the failure and keeps running.

For example: If shard0 got failed, then still our system can continue to run safely with help of other shards but it won't be able to query the failed shard. That means it has fault tolerance. Fault tolerance is related to CAP but not part of it.

Note:

Partition tolerance talks about the failure of communication between shards but fault tolerance talks about failure of shards itself.

Hence, any distributed database in this real world can have only two of these properties. Generally, a database sacrifices either of A or C as P is one of the most important property of the sharding.

So, this was all about to know background of what is CAP and fault tolerance. Now let's talk about our sharded database:

→ Consistency in our Sharded Database:

In our database, consistency is maintained within each shard. Each shard runs an independent instance of MySQL, which ensures strong consistency through ACID-compliant transactions which our API also guarantees from Assignment – 3.

Since the database is horizontally sharded using the partitioning rule:

$$\text{Shard} = \text{Student ID} \% 3$$

And hence, each student's record is stored in exactly one shard. When attendance for a student is updated, the update is performed on the specific shard responsible for that student.

Because the application always routes queries to the correct shard using this deterministic rule, the latest data is returned for that student record. Therefore, consistency is preserved within each shard.

Note:

Consistency generally breaks when Availability is chosen over it. Since, we'll see ahead that our system lacks availability and hence this ensures that our system is in most of times is consistent. It won't break often.

But still good systems can lose consistency due to:

- Bugs in routing logic
- Incorrect shard mapping
- Failed transactions
- Application errors

→ Partition Tolerance in our Sharded Database:

In our sharded database, each shard operates as an independent instance, i.e., one shard is not related to another as both store very independent data.

This helps our system to have partition tolerance as well as fault tolerance. This is because if communication between two shards is disrupted due to network issues, the shards can still continue processing requests related to the data they store as they are independent. Since query routing is handled by the application layer, the system can still interact with the available shards even if one shard becomes unreachable.

This ensures that the overall system continues operating despite network failures affecting individual shards.

In addition to it, if a shard fails completely, then still our database system can work using other shards due to independency. Since, we don't have backup or store data of a shard in another shard, if one shard gets failed, we won't

have any data and hence won't be able to respond to the query but still our database will work for other users not in the shard.

Therefore, since our system fulfils **Consistency** and **Partition Tolerance**, we can say that our architecture is a **CP System**.

→ Availability in our Database:

This is the **tradeoff** that we do in our CP system. Our system **lacks availability**. This is because we have only three shards and all independent of each other. We have not also any backups of shards that store old data nor any mechanism to response to query if shard isn't there.

Hence, if a shard fails in our database system, then those students of that shard when ask for query, our system would result in error because once shard is gone, we can't respond them. Requests won't be able to serve until the shard is restored.

For example: Shard 1 failed and a student of that shard asks for attendance. Though with help of member table he/she could login but when they ask for attendance in a course, our system would result in error or Data Not found.

So, by this way our system can handle Consistency and Partition Tolerance but it lacks Availability.

Note:

A distributed database system can never claim; his system is CA because a CA system can never be distributed! It has to be CP or AP. This is because in practical distributed systems, partition tolerance is usually considered essential because network failures are unavoidable. Therefore, most distributed databases choose either CP or AP behavior during partitions.

Hence, our system is CP system with tradeoff of Availability.

□ What happens to your system if one shard goes down?

If one shard goes down, then since our system is CP:

- **Partition Tolerance:** The system continues to run using the other shards. Because each shard is independent, the failure of one doesn't collapse the whole database. That's partition tolerance. Hence, our database will continue to run.
- **Availability:** Students whose data is in the failed shard cannot access attendance. They can still log in (since the member table is global), but when they query attendance, the system returns an error. This shows availability is sacrificed. So those students would be ghosted until the server is brought back online.
- **Fault Tolerance:** Limited — since there are no replicas, the failed shard cannot be recovered automatically. But still the database would be able to run properly with help of other shards. So until the server of failed shard is brought back online, we won't be able to do anything for that.

Hence, at cost of our database being always active, we allowed our database to atleast run well even if a network breaks. Hence, we can say:

“Our Database System is fighter – it will lose a limb but will still fight!”

□ How does your design handle a simulated shard failure?

Because shards are independent and there are no backups, the failed shard cannot be recovered automatically. The system handles this by continuing to serve unaffected shards, while returning errors for queries directed at the failed shard.

This demonstrates CP behavior: consistency and partition tolerance are preserved, availability is sacrificed.

While we lack automated real-time availability (failover), recovery in our design is handled through Manual Intervention: restoring the failed MySQL instance on its dedicated port (e.g., 8081) and verifying the shard key routing once the service is back online. Hence, we will have to manually go and restore the shard.

Part – IV: Conclusion

✿ Tradeoffs & Scalability Results:

So, time to conclude our results! So in our database system was designed using horizontal sharding, where students were recorded across multiple database instances based on the shard key that we chose: `student_id%3`. This helps us to spread the data uniformly across shards and hence reduce the load on a single server. By implementing this Shared-Nothing Architecture, we ensure that none of the shards share data or resources, effectively spreading the computational load uniformly and preventing any single server from becoming a performance bottleneck.

As the number of students increases, the workload is equally divided among the shards. Instead of a single database handling all queries, each shard is responsible for a subset of the students depending on the hash function that we used to partition. This improves scalability because queries related to different students can be processed independently by different shards. Hence, by this way our database can now handle large datasets.

However, this architecture introduces certain tradeoffs some of which are expected. While sharding improves scalability and distributes workload effectively, it also increases system complexity. Query routing must be handled carefully by the application layer to ensure that requests reach the correct shard.

Additionally, the system sacrifices availability during shard failures and this is expected as from CAP's theorem it is impossible for a distributed database system to have consistency, availability as well as partition tolerance. Hence, we trade off Availability for Consistency and Partition Tolerance, which makes our system CP System.

Since the shards do not replicate data from one another, if a shard becomes unavailable, the system cannot serve requests related to the data stored in that shard and for recovery, we will have to manually bring the shard online. Hence, that's why it lacks availability.

Thus, while the system scales better with increasing data size, it introduces tradeoffs in terms of fault recovery and availability.

✿ Observations & Limitations:

Some of the observations and Limitations of our Database Sharding are:

1. **Data Distribution Success:** Our verification of the migration showed a near-perfect distribution (79, 201, and 105 records). This confirms that Student ID is a high-cardinality shard key, preventing "Hot Spots" where one server does all the work while others stay idle. This data gets better as real data is used. Since here we used random data like 230005, 230011, that means not consecutive, that's why we could see such uneven split. But as we go more towards real data, using consecutive ids, this unevenness decreases and equal distribution takes place.
2. **The Ghosting Effect:** Through this assignment, we observed the challenges of applying all three: CAP in one sharded database. We tried to apply Consistency, Partition Tolerance as well as availability but failed to do so. Adding consistency clashed with that of Availability as one doesn't allow wrong values but one can allow. Hence, we decided to stay with the CP system and let tradeoff availability. So, if a shard fails, then the student of that shard can't access his data and will be shown error. It would be as if it doesn't exist. This is ghosting effect.
3. **Cross-Shard Query Latency:** We observed that queries involving ranges or global statistics (like overall university averages) are more resource-intensive. The application must perform an "Aggregate-and-Merge" operation, which is a common limitation in distributed sharding.

So, this was all about the Sharding in the Attendance Management Portal.

